SlipSurfaceIQ LE

Python Scripting Tutorial

Build and edit complete slope models with Python

Version 2026.6.0

Contents

01 Getting started . . . 3
02 Names available in a script . . . 4
03 Project summary . . . 5
04 Geometry . . . 5
05 Materials . . . 6
06 Groundwater and tension cracks . . . 10
07 External loads . . . 11
08 Supports . . . 11
09 Facings . . . 13
10 Seismic loading . . . 14
11 Analysis methods and settings . . . 14
12 Slip surfaces . . . 16
13 Notes and display options . . . 17
14 A complete model . . . 19
15 Troubleshooting . . . 20

01 Getting started

Python scripting builds or edits a complete model in a few lines: geometry, materials, water, loads, supports, settings and notes.

Open the editor with **Tools → Python scripting**. It opens empty. Write a script, or open a **.py** file, then press **Run / Apply**. **Save .py** stores the editor text; **Tutorial** opens this document.

How a script runs

• The script receives **project**, a *copy* of the open model. Change it in place, or assign a new project to the same name.

• When the script finishes, the result is validated and its geometry is built. If everything is valid, all changes are applied as **one undoable step** (Ctrl+Z in the Modeler reverts the whole script).

• If the script raises an error or the model is invalid, the open project is left unchanged and the message appears in the output pane.

• Anything printed with **print()** is shown in the output pane.

Units
Lengths in m, forces in kN, stresses in kPa, unit weights in kN/m^3, angles in degrees. The y axis points up.

Trusted code only
Scripts are ordinary Python with your account's permissions. A script opened from a file asks for confirmation before its first run. Projects never run scripts on their own.

Your first script

first_script.py

```
from slipsurface.project import empty_project, MaterialSpec

project = empty_project()
project.summary.title = "My first slope"
project.external = [[0, -10], [60, -10], [60, 10], [40, 10], [20, 0], [0, 0]]
project.materials = [MaterialSpec(name="Clay", gamma=19, c=8, phi=26)]
project.methods.methods = ["bishop", "spencer"]
print("Materials:", project.material_names())
```

Press **Run / Apply**; the slope appears in the Modeler and is ready for **Compute**.

02 Names available in a script

Name	What it is
`project`	Copy of the open model (Project). The result is what it holds at the end.
`empty_project()`	New project with one default material 'Soil 1' and no geometry.
`Project`	The project class; Project() has no materials.
`MaterialSpec(...)`	A material: name, model and parameters.
`SupportType(...)`	A support definition (anchor, nail, geosynthetic, …).
`FacingType(...)`	A shotcrete + mesh facing definition.
`print(...)`	Write to the output pane.

Everything else in Python works as usual: loops, functions, the **math** module and imports from the application, e.g. `from slipsurface.loads import SUPPORT_KINDS`.

Useful project helpers

Call	Returns
`project.name`	The title (same as project.summary.title).
`project.material_names()`	List of material names.
`project.material(name)`	The MaterialSpec with that name, or None.
`project.support_type(name)`	The SupportType with that name, or None.
`project.facing_type(name)`	The FacingType with that name, or None.
`project.validate()`	Raise ValueError if the model is inconsistent.
`project.copy()`	An independent deep copy.

03 Project summary

Shown in the window title and on the report cover.

Field	Default	Meaning
`title`	`'New Project'`	Project title (window caption, report cover).
`analysis`	`''`	Analysis description.
`author`	`''`	Author shown in the report.
`company`	`''`	Company shown in the report.
`date_created`	`today`	Creation date, YYYY-MM-DD.
`comments`	`['', '', '', '', '']`	Five free comment lines.

summary.py

```
project.summary.title = "Highway cut, km 12+400"
project.summary.author = "H. Engineer"
project.summary.company = "Geotech Ltd."
project.summary.comments[0] = "Long-term drained analysis"
```

04 Geometry

Points are [x, y] lists in metres. The external boundary is one closed polygon; material boundaries split it into regions.

Field	Format	Meaning
`external`	`[[x, y], ...]`	External boundary polygon, at least 3 points (closed automatically).
`material_boundaries`	`[[[x, y], ...], ...]`	Polylines that divide the domain into regions.
`assignments`	`[{'point': [x, y], 'material': name}]`	A point inside a region gives it a material. Unassigned regions use the first material.

A layered slope

layers.py

```
project.external = [[0, -12], [70, -12], [70, 12], [45, 12], [25, 0], [0, 0]]
project.materials = [
    MaterialSpec(name="Fill", gamma=19, c=5, phi=32, color="#f0c800"),
    MaterialSpec(name="Clay", gamma=18, c=12, phi=22, color="#3cb371"),
]
project.material_boundaries = [[[0, -4], [70, -4]]]
project.assignments = [
    {"point": [50, 5], "material": "Fill"},
    {"point": [50, -8], "material": "Clay"},
]
```

Build boundaries with loops
Geometry is plain data, so loops and functions work well — for example, several horizontal layers from a list of elevations.

layers_loop.py

```
tops = [-3, -7]                      # layer boundary elevations
names = ["Sand", "Silt", "Clay"]
project.materials = [MaterialSpec(name=n, c=5 + 5 * i, phi=32 - 4 * i)
                     for i, n in enumerate(names)]
project.material_boundaries = [[[0, y], [60, y]] for y in tops]
mids = [-1.5, -5, -9]                # a point inside each layer
project.assignments = [{"point": [50, y], "material": n}
                       for y, n in zip(mids, names)]
```

05 Materials

Every material has common fields and the parameters of its strength model.

Field	Default	Meaning
`name`	`'Soil 1'`	Unique material name (assignments refer to it).
`model`	`'MC'`	Strength model key, see the models table.
`gamma`	`19.0`	Unit weight above the water table (kN/m3).
`gamma_sat`	`20.0`	Saturated unit weight (kN/m3), used when use_sat is True.
`use_sat`	`False`	Use gamma_sat below the water table.
`use_water`	`True`	The water table acts on this material.
`hu`	`1.0`	Pore-pressure factor Hu applied to the water table head.
`color`	`'#f0c800'`	Fill colour, e.g. '#f0c800'.

Strength models

Set **model** to one of the keys below and give the parameters listed for it; unused parameters are ignored.

model	Strength model	Parameters
`MC`	Mohr-Coulomb	`c, phi`
`UD`	Undrained (Tresca)	`su`
`UDG`	Undrained – linear with elevation	`su_top, su_bottom, su_y_top, su_y_bottom`
`SH`	SHANSEP	`shansep_s, shansep_m, ocr, shansep_su_min`
`HS`	Mohr-Coulomb (HS envelope)	`c, phi, hs_psi, hs_e50, hs_eoed, hs_eur, hs_m, hs_pref, hs_rf`
`SS`	Approximate peak ϕ (SANISAND)	`ss_m, ss_e, ss_ec0, ss_lambda, ss_xi, ss_nb, ss_pat`
`PL`	Power-law envelope	`coefficient, exponent, reference_pressure`
`PC`	Power Curve	`pc_a, pc_b, pc_c, pc_d`
`BB`	Barton-Bandis	`bb_phir, bb_jrc, bb_jcs, bb_l0, bb_ln`

model	Strength model	Parameters
`CN`	CNI Rock Mass	`cni_sigma_c, cni_a, cni_n, cni_t`
`HB`	Generalized Hoek-Brown	`sigma_ci, gsi, mi, d`
`HBD`	Hoek-Brown (mb, s, a)	`sigma_ci, hb_mb, hb_s, hb_a`

Mohr-Coulomb (MC)

τ = c′ + σ′n · tan ϕ′

Parameter	Default	Meaning
`c`	`5.0`	Cohesion c (kPa)
`phi`	`30.0`	Friction angle ϕ (°)

Undrained (Tresca) (UD)

τ = su (total stress; pore pressure not used)

Parameter	Default	Meaning
`su`	`25.0`	Undrained shear strength su (kPa)

Undrained – linear with elevation (UDG)

τ = su(y), linear from layer top to bottom (total stress)

Parameter	Default	Meaning
`su_top`	`25.0`	Undrained strength at the top elevation (kPa)
`su_bottom`	`25.0`	Undrained strength at the bottom elevation (kPa)
`su_y_top`	`0.0`	Top elevation (m)
`su_y_bottom`	`-1.0`	Bottom elevation (m)

SHANSEP (SH)

τ = su = A + S · σ′v · OCR^m

Parameter	Default	Meaning
`shansep_s`	`0.22`	S (normally consolidated su/σ′v)
`shansep_m`	`0.8`	m (exponent)
`ocr`	`1.0`	OCR (overconsolidation ratio)
`shansep_su_min`	`0.0`	A — minimum undrained strength (kPa)

Mohr-Coulomb (HS envelope) (HS)

τ = c′ + σ′n · tan ϕ′ (Mohr-Coulomb failure surface)

Parameter	Default	Meaning
`c`	`5.0`	Cohesion c′ (kPa)
`phi`	`30.0`	Friction angle ϕ′ (°)

Parameter	Default	Meaning
`hs_psi`	`0.0`	Dilatancy angle ψ (°)
`hs_e50`	`30000.0`	E50,ref (kPa)
`hs_eoed`	`30000.0`	Eoed,ref (kPa)
`hs_eur`	`90000.0`	Eur,ref (kPa)
`hs_m`	`0.5`	m (stress dependency exponent)
`hs_pref`	`100.0`	pref (kPa)
`hs_rf`	`0.9`	Rf (failure ratio)

Approximate peak ϕ (SANISAND) (SS)

τ = σ′n tan ϕ(ψ), sin ϕ = 3Mb/(6+Mb), Mb = M·exp(−nb ψ)

Parameter	Default	Meaning
`ss_m`	`1.25`	M (critical state stress ratio)
`ss_e`	`0.75`	Void ratio e
`ss_ec0`	`0.934`	ec0 (CSL reference void ratio)
`ss_lambda`	`0.019`	λc
`ss_xi`	`0.7`	ξ
`ss_nb`	`1.1`	nb
`ss_pat`	`101.3`	pat (kPa)

Power-law envelope (PL)

τ = a · pa · (1 + σ′n/pa)^b

Parameter	Default	Meaning
`coefficient`	`0.2`	a
`exponent`	`0.5`	b
`reference_pressure`	`100.0`	pa (kPa)

Power Curve (PC)

τ = a · (σ′n + d)^b + c

Parameter	Default	Meaning
`pc_a`	`1.5`	a
`pc_b`	`0.8`	b
`pc_c`	`0.0`	c (kPa)
`pc_d`	`0.0`	d (kPa)

Barton-Bandis (BB)

τ = σ′n · tan(ϕr + JRCn · log10(JCSn/σ′n))

Parameter	Default	Meaning
bb_phir	25.0	Residual friction angle ϕr (°)
bb_jrc	10.0	JRC0
bb_jcs	50000.0	JCS0 (kPa)
bb_l0	0.1	Laboratory sample length L0 (m)
bb_ln	0.1	Field block size Ln (m)

CNI Rock Mass (CN)

τ = a · σc · ((σn − T0) / σc)^n

Parameter	Default	Meaning
cni_sigma_c	10000.0	Rock mass compressive strength σc (kPa)
cni_a	0.6	Strength magnitude coefficient a
cni_n	0.75	Curvature coefficient n
cni_t	100.0	Direct tensile strength |T0| (kPa)

Generalized Hoek-Brown (HB)

σ1 = σ3 + σci (mb σ3/σci + s)^a (generalized Hoek-Brown, 2002)

Parameter	Default	Meaning
sigma_ci	25000.0	Intact UCS σci (kPa)
gsi	50.0	GSI
mi	10.0	mi
d	0.0	Disturbance factor D

Hoek-Brown (mb, s, a) (HBD)

σ1 = σ3 + σci (mb σ3/σci + s)^a

Parameter	Default	Meaning
sigma_ci	25000.0	Intact UCS σci (kPa)
hb_mb	1.0	mb
hb_s	0.001	s
hb_a	0.5	a

Examples

materials.py

```
project.materials = [
    MaterialSpec(name="Sand", model="MC", gamma=19, c=0, phi=34),
    MaterialSpec(name="Soft clay", model="UD", gamma=17, su=25, use_water=False),
    MaterialSpec(name="Clay", model="SH", gamma=18,
                 shansep_s=0.25, shansep_m=0.8, ocr=2),
    MaterialSpec(name="Rock", model="HB", gamma=24,
                 sigma_ci=20000, gsi=45, mi=10, d=0),
]
project.assignments = [{"point": [50, 5], "material": "Sand"}]
```

06 Groundwater and tension cracks

Field	Format	Meaning
`water_table`	`[[x, y], ...] or None`	Piezometric line across the model.
`tension_crack`	`[[x, y], ...] or None`	Line whose depth below the ground defines the crack (at least 2 points, increasing x).

project.groundwater

Field	Default	Meaning
`method`	`'water_table'`	water_table, ru or none.
`gamma_w`	`9.81`	Unit weight of water (kN/m3).
`ru`	`0.0`	Pore pressure coefficient Ru (0..1) when method = ru.
`crack_water`	`0.0`	Tension crack water filling, 0 (dry) .. 1 (full).

water.py

```
project.water_table = [[0, -2], [20, -2], [45, 6], [60, 6]]
project.groundwater.method = "water_table"
project.materials[0].hu = 1.0            # per-material head factor
project.tension_crack = [[45, 7], [60, 7]]   # 3 m deep below the crest
project.groundwater.crack_water = 0.5        # half full
```

ru.py

```
project.groundwater.method = "ru"
project.groundwater.ru = 0.25
```

07 External loads

project.loads is a list of dictionaries.

Key	Value	Meaning
`kind`	`'distributed'`	Distributed load between two points.
`points`	`[[x1, y1], [x2, y2]]`	Start and end on the surface (x1 ≠ x2).
`q`	`[q1, q2]`	Intensity at the two ends (kPa), linear in between.
`orientation`	`'vertical' | 'normal'`	Load direction (default vertical).
`kind`	`'line'`	Point (line) load.
`point`	`[x, y]`	Where it acts.
`magnitude`	`number`	kN/m.
`angle`	`degrees`	Direction from +x, counter-clockwise; 270 = downward (default).

loads.py

```
project.loads = [
    {"kind": "distributed", "points": [[45, 10], [55, 10]],
     "q": [20, 20], "orientation": "vertical"},
    {"kind": "line", "point": [50, 10], "magnitude": 60, "angle": 270},
]
```

08 Supports

Define support types once, then place single supports or automatic bolt patterns that refer to them by name.

kind	Model
`end_anchored`	End anchored
`grouted_tieback`	Grouted tieback
`soil_nail`	Soil nail
`geotextile`	Geotextile
`geosynthetic`	Geosynthetic
`micro_pile`	Micro-pile

SupportType fields

Field	Default	Meaning
`name`	`'Anchor 1'`	Unique name; supports and patterns refer to it.
`kind`	`'end_anchored'`	Support model (see the support kinds table).
`spacing`	`1.5`	Out-of-plane spacing (m); not used for geotextile / geosynthetic.
`tensile`	`200.0`	Tensile capacity: kN, or kN/m for geotextile / geosynthetic.
`plate`	`100.0`	Plate capacity of a soil nail (kN).
`bond_strength`	`50.0`	Bond strength of tiebacks and soil nails (kN/m).
`bond_length`	`4.0`	Bonded length of a grouted tieback (m).
`pullout`	`20.0`	Geotextile pull-out strength (kN/m per m).
`application`	`'active'`	active (force reduces driving) or passive (force divided by F).
`orientation`	`'parallel'`	parallel, tangent or perpendicular force direction.
`color`	`'#1f3d7a'`	Drawing colour.
`length`	`12.0`	Default length of automatically placed bolts (m).
`coverage`	`100.0`	Geosynthetic strip coverage (%).
`anchorage`	`'none'`	Geosynthetic anchorage: none, face, embedded or both.
`adhesion`	`0.0`	Geosynthetic interface adhesion (kPa).
`interface_friction`	`25.0`	Geosynthetic interface friction angle (deg).
`overburden_gamma`	`19.0`	Unit weight for the geosynthetic overburden stress (kN/m3).
`pile_shear`	`100.0`	Shear strength of one micro-pile (kN).

Single supports — project.supports

Each item is `{'a': [x, y], 'b': [x, y], 'type': name}`: **a** is the head on the slope face, **b** the end inside the slope.

Bolt patterns — project.support_patterns

Key	Meaning
`p1, p2`	[x, y] points; bolts are placed on the ground between their x values.
`spacing`	Distance between bolts along the slope (m, > 0).
`offset`	Distance of the first bolt from p1 (m, default 0).
`angle`	Inclination below horizontal (deg, default 15).
`length`	Bolt length (m, > 0).
`type`	Name of a SupportType.

supports.py

```
project.support_types = [
    SupportType(name="Tieback", kind="grouted_tieback", spacing=2, tensile=300,
                bond_strength=60, bond_length=5, application="active"),
    SupportType(name="Nail", kind="soil_nail", spacing=1.5, tensile=200,
                plate=80, bond_strength=40, application="passive", length=9),
]
project.supports = [{"a": [30, 5], "b": [42, 1], "type": "Tieback"}]
project.support_patterns = [
    {"p1": [20, 0], "p2": [40, 10], "spacing": 2, "offset": 1,
     "angle": 15, "length": 9, "type": "Nail"},
]
```

geosynthetic.py

```
project.support_types = [
    SupportType(name="Geogrid", kind="geosynthetic", tensile=80, coverage=100,
                anchorage="embedded", interface_friction=28),
    SupportType(name="Pile", kind="micro_pile", spacing=1.2, pile_shear=150),
]
project.supports = [
    {"a": [22, 1], "b": [55, 1], "type": "Geogrid"},
    {"a": [35, 7.5], "b": [35, -6], "type": "Pile"},
]
```

09 Facings

Shotcrete with welded wire mesh on the slope face.

Field	Default	Meaning
`name`	`'Shotcrete'`	Unique name; facings refer to it.
`thickness`	`0.1`	Shotcrete thickness (m).
`gamma`	`24.0`	Shotcrete unit weight (kN/m3).
`tau`	`400.0`	Shotcrete shear strength (kPa).
`mesh`	`'Q188'`	Mesh designation, e.g. Q188.
`mesh_as`	`188.0`	Mesh steel area (mm2/m).
`mesh_fy`	`500.0`	Mesh yield strength (MPa).
`application`	`'passive'`	passive (T/F) or active.
`color`	`'#7f8c8d'`	Drawing colour.

Capacity per metre of slope = τ · thickness + As · fy. Place a facing with `{'x1': x, 'x2': x, 'type': name, 'points': [[x1, y1], [x2, y2]]}` in **project.facings**.

facing.py

```
project.facing_types = [FacingType(name="Shotcrete 10 cm", thickness=0.10,
                                   tau=400, mesh="Q188", mesh_as=188, mesh_fy=500)]
project.facings = [{"x1": 20, "x2": 40, "type": "Shotcrete 10 cm",
                    "points": [[20, 0], [40, 10]]}]
```

10 Seismic loading

Field	Default	Meaning
kh	0.0	Horizontal seismic coefficient (acts in the failure direction).
kv	0.0	Vertical coefficient; positive is upward: load (1 - kv) W.
both_vertical	True	Also solve -kv and keep the lower factor of safety.

Sign of kv
Positive kv is upward, so the vertical load is (1 − kv)·W. In Slide2 a positive kv acts downward. With both_vertical = True both signs are solved and the lower factor of safety is kept.

seismic.py

```
project.seismic.kh = 0.15
project.seismic.kv = 0.05
```

11 Analysis methods and settings

Key	Method	Equilibrium
bishop	Bishop simplified	moment
corps1	Corps of Engineers #1	force
corps2	Corps of Engineers #2	force
gle	GLE/Morgenstern-Price	force and moment
janbu_simplified	Janbu simplified	force
janbu_corrected	Janbu corrected	force (corrected)
lowe_karafiath	Lowe-Karafiath	force
ordinary	Ordinary/Fellenius	moment
spencer	Spencer	force and moment

project.methods

Field	Default	Meaning
`methods`	`[…]`	Limit equilibrium methods to run (keys in the methods table).
`n_slices`	`50`	Number of slices (at least 3).
`tolerance`	`0.005`	Convergence tolerance on the factor of safety.
`max_iter`	`75`	Maximum solver iterations.
`interslice`	`'half-sine'`	GLE interslice function: half-sine, constant, clipped-sine, trapezoidal.
`f_start`	`0.5`	Trapezoidal / clipped function value at the start (0..1).
`f_end`	`0.5`	Trapezoidal / clipped function value at the end (0..1).

project.general

Field	Default	Meaning
`units`	`'metric'`	Unit system; metric (m, kN, kPa).
`failure_direction`	`'right_to_left'`	auto, right_to_left or left_to_right.
`data_output`	`'standard'`	Amount of result data kept: standard.

project.advanced

Field	Default	Meaning
`check_admissibility`	`True`	Reject surfaces with inadmissible slice forces.
`min_depth`	`0.0`	Minimum slide depth (m).
`m_alpha_min`	`0.2`	Admissibility threshold for m-alpha (0..1].
`e_max_ratio`	`50.0`	Reject interslice forces above ratio x total weight.

methods.py

```
project.methods.methods = ["bishop", "spencer", "gle"]
project.methods.n_slices = 40
project.methods.interslice = "half-sine"
project.general.failure_direction = "right_to_left"
project.advanced.min_depth = 1.0
```

12 Slip surfaces

project.surfaces selects the surface type and the search.

Field	Default	Meaning
surface_type	'circular'	circular, noncircular or user.
search	'auto_refine'	Search strategy: auto_refine.
effort	'normal'	Search effort: fast, normal, fine.
n_ctrl	4	Interior vertices of non-circular surfaces.
noncircular_shape	'polyline'	polyline (straight segments) or spline.
convex_only	True	Keep non-circular surfaces convex.
seed	0	Random seed of the search (reproducible results).
left_range	None	Slope limits for the left end: [x_min, x_max] or None.
right_range	None	Slope limits for the right end: [x_min, x_max] or None.
user_surface	None	User polyline [[x, y], ...] when surface_type = user.
user_circle	None	Exact fixed circle [xc, yc, radius] when surface_type = user.

surfaces.py

```
# automatic non-circular search with slope limits
project.surfaces.surface_type = "noncircular"
project.surfaces.effort = "normal"
project.surfaces.left_range = [10, 25]
project.surfaces.right_range = [40, 60]
```

fixed_surface.py

```
# analyse one given circle only
project.surfaces.surface_type = "user"
project.surfaces.user_circle = [25, 30, 30]   # xc, yc, radius
```

13 Notes and display options

Notes on the drawing — project.annotations

Key	Meaning
`kind`	text, line, arrow, rect or ellipse.
`pts`	[[x, y]] for text; [[x1, y1], [x2, y2]] for the other shapes.
`text`	Text content (text notes); '\n' starts a new line.
`color`	Line or text colour, e.g. '#c0392b'.
`width`	Line width.
`fill`	Fill colour of rect / ellipse, or None.
`size, bold, box`	Text size, bold text and frame around the text.

notes.py

```
project.annotations = [
    {"kind": "text", "pts": [[30, 16]], "text": "Design section A-A",
     "color": "#1b2a3a", "size": 11, "bold": True, "box": True},
    {"kind": "arrow", "pts": [[30, 15], [26, 3]], "color": "#c0392b", "width": 1.5},
]
```

project.display

Key	Default	Meaning
external_color	'#606060'	External boundary colour.
material_boundary_color	'#808080'	Material boundary colour.
water_table_color	'#1f5fd1'	Water table colour.
show_vertices	True	Draw boundary vertices.
line_width	2	Boundary line width.
show_ruler	True	Show the rulers.
show_grid	True	Show the grid.
fill_materials	True	Fill regions with material colours.
show_slope_limits	True	Show the slope limit markers.
show_slices	False	Draw the slices of the critical surface.
slice_fill_color	'#f6d65a'	Slice fill colour.
show_lines_to_center	True	Lines from the critical circle to its centre.
show_fs_text	True	Write the factor of safety at the centre.
show_all_surfaces	False	Draw the lowest surfaces, not only the critical one.
show_fs_contours	True	Slip-centre FS contour map (circular searches).
critical_dashed	True	Critical surface as a dashed line.
show_legend	True	Material legend.
show_fs_legend	True	FS legend of all methods.
material_labels	'none'	Region labels: none, name or full.
label_size	8	Label font size.
show_control_points	False	Control polygon of a spline critical surface.

display.py

```
project.display["material_labels"] = "full"
project.display["show_slices"] = True
```

14 A complete model

Everything together: copy, run, then Compute.

complete_model.py

```
from slipsurface.project import empty_project, MaterialSpec
from slipsurface.loads import SupportType, FacingType

project = empty_project()
project.summary.title = "Scripted layered cut"
project.external = [[0, -8], [48, -8], [48, 12], [26, 12], [20, 0], [0, 0]]
project.materials = [
    MaterialSpec(name="Soil", model="MC", gamma=19, c=8, phi=30),
    MaterialSpec(name="Rock", model="HB", gamma=23, sigma_ci=8000, gsi=40, mi=8),
]
project.material_boundaries = [[[0, -3], [48, -3]]]
project.assignments = [
    {"point": [24, 5], "material": "Soil"},
    {"point": [24, -6], "material": "Rock"},
]
project.water_table = [[0, -2], [20, -2], [40, 4], [48, 4]]
project.tension_crack = [[30, 10], [48, 10]]
project.groundwater.crack_water = 0.5
project.support_types = [
    SupportType(name="Nail", kind="soil_nail", spacing=1.5, tensile=240,
                plate=80, bond_strength=40, application="passive", length=9),
]
project.support_patterns = [
    {"p1": [20, 0], "p2": [26, 12], "spacing": 2, "offset": 1,
     "angle": 15, "length": 9, "type": "Nail"},
]
project.facing_types = [FacingType(name="Shotcrete")]
project.facings = [{"x1": 20, "x2": 26, "type": "Shotcrete",
                    "points": [[20, 0], [26, 12]]}]
project.loads = [
    {"kind": "distributed", "points": [[29, 12], [40, 12]],
     "q": [15, 15], "orientation": "vertical"},
    {"kind": "line", "point": [42, 12], "magnitude": 20, "angle": 270},
]
project.seismic.kh = 0.1
project.methods.methods = ["bishop", "spencer"]
project.methods.n_slices = 40
project.surfaces.effort = "fast"
print("Ready:", project.summary.title)
```

15 Troubleshooting

Message	Cause and fix
`Define at least one material.`	project.materials is empty.
`Material names must be unique.`	Two materials share a name.
`Assigned material 'X' is not defined.`	An assignment or the material list has a typo.
`Support type 'X' is not defined.`	A support or pattern names a missing SupportType.
`Unknown method: X`	Use a key from the methods table.
`unknown option 'X'`	A choice field has a value that is not allowed; see the tables.
`No external boundary defined`	project.external needs at least 3 points.
`project must be a Project instance`	The name project was reassigned to something else.

Debugging
Use print() freely, and call project.validate() inside the script to see a validation error at the line where you made the change.